



UNIVERSITÀ DEGLI STUDI DI GENOVA

DIBRIS

DEPARTMENT OF INFORMATICS,
BIOENGINEERING, ROBOTICS AND SYSTEM ENGINEERING

MODELLING AND CONTROL OF MANIPULATORS

Third Assignment

Jacobian Matrices and Inverse Kinematics

Author:

Beaini Charbel
Gjinaj Endri
Topulli Joel

Student ID:

S8027823
S8663382
S5418701

Professors:

Enrico Simetti
Giorgio Cannata

Tutors:

Luca Tarasi
Simone Borelli

January 6, 2026

Contents

1	Assignment description	3
1.1	Exercise 1	3
1.2	Exercise 2	3
2	Exercise 1	5
3	Exercise 2	6
3.1	Q2.1	6
3.2	Q2.2	7
3.3	Q2.3	8
3.4	Q2.4	9
3.5	Q2.5	10

Mathematical expression	Definition	MATLAB expression
$\langle w \rangle$	World Coordinate Frame	w
${}^a_b R$	Rotation matrix of frame $\langle b \rangle$ with respect to frame $\langle a \rangle$	aRb
${}^a_b T$	Transformation matrix of frame $\langle b \rangle$ with respect to frame $\langle a \rangle$	aTb
${}^a O_b$	Vector defining frame $\langle b \rangle$ with respect to frame $\langle a \rangle$	aOb

Table 1: Nomenclature Table

1 Assignment description

The third assignment of Modelling and Control of Manipulators focuses on Inverse Kinematics (IK) control of a robotic manipulator.

The third assignment consists of three exercises. You are asked to:

- Download the .zip file called from the Aulaweb page of this course.
- Implement the code to solve the exercises on MATLAB by filling in the predefined files.
- Write a report motivating your answers, following the predefined format on this document.
- **Do NOT put your code in the report!**

1.1 Exercise 1

Given the geometric model of an industrial manipulator in Figure 1, you have to add a tool frame. The tool frame is rigidly attached to the robot end-effector according to the following specifications: $\Gamma_{t/e} = [\pi/10, 0, \pi/6]$, ${}^eO_t = [0.3, 0.1, 0]^T (m)$ where $\Gamma_{t/e}$ represents the YPR values from end effector frame to tool frame.

To complete this task you should modify the class *geometricModel* by adding a new method called *getToolTransformWrtBase*.

1.2 Exercise 2

Implement an inverse kinematic control loop to control the tool of the manipulator. You should be able to complete this exercise by using the MATLAB classes implemented for the previous assignment (*geometricModel*, *kinematicModel*), and also you need to implement a new class *cartesianControl* (see the template attached). The initial joint position is $q = [\pi/2, -\pi/4, 0, -\pi/4, 0, 0.15, \pi/4]^T$, expressed in radians and meters. The procedure can be split into the following phases

Q2.1 Compute the cartesian error between the robot end-effector frame bT and the goal frame bT_g .

The goal frame must be defined knowing that:

- The goal position with respect to the base frame is ${}^bO_g = [0.2, -0.7, 0.3]^T (m)$
- The goal frame is rotated by the YPR angles $\Gamma_g = [0, 1.57, 0]^T (rad)$ with respect to the base frame.

Q2.2 Compute the desired angular and linear reference velocities of the tool with respect to the base:

$${}^b\nu_{t/b}^* = \begin{bmatrix} \kappa_a & 0 \\ 0 & \kappa_l \end{bmatrix} \cdot {}^b e, \text{ such that } \kappa_a = 0.8, \kappa_l = 0.8 \text{ is the gain.}$$

Q2.3 Compute the desired joint velocities $\dot{q}$

Q2.4 Simulate the robot motion by implementing the function: "*KinematicSimulation()*" for integrating the joint velocities in time.

Q2.5 Compute the end effector and tool linear and angular velocities with respect to the base frame projected on the base frame.

Remark 1: All the methods must be implemented for a generic serial manipulator. For instance, joint types, and the number of joints should be parameters.

Remark 2: The class *plotManipulators* is provided for plot generation. You must not modify its implementation, or its calls in *main.m*.

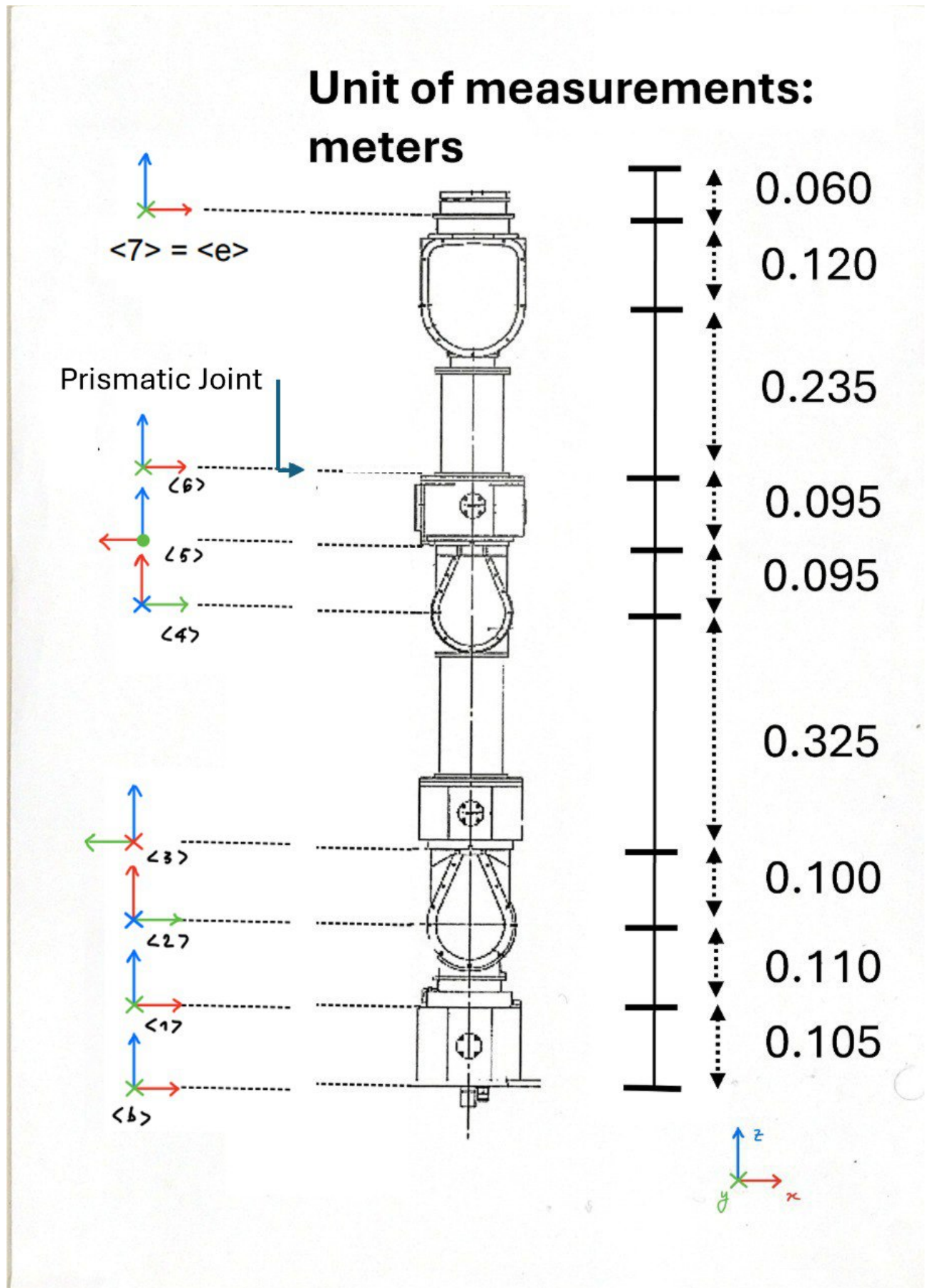


Figure 1: Manipulator CAD

2 Exercise 1

This exercise extends the geometric model of the assigned 7-DoF serial manipulator by introducing a rigid tool frame $\langle t \rangle$ attached to the end-effector frame $\langle e \rangle$. The complete robot model is therefore described by the standard set of relative homogeneous transformations ${}^i{}_{i-1}T(q_i)$, augmented by a constant transform ${}^e{}_tT$ that captures the tool offset with respect to the end-effector.

A homogeneous transformation has the form

$${}^a{}_bT = \begin{pmatrix} {}^a{}_bR & {}^a r_{a/b} \\ 0 & 1 \end{pmatrix}, \quad (1)$$

where ${}^a{}_bR \in \mathbb{R}^{3 \times 3}$ is a rotation matrix and ${}^a r_{a/b} \in \mathbb{R}^{3 \times 1}$ is the translation vector.

The tool pose with respect to the end-effector is assigned by a translation

$${}^e r_{e/t} = \begin{bmatrix} 0.3 \\ 0.1 \\ 0 \end{bmatrix} \text{ m},$$

and by yaw-pitch-roll angles

$$\Gamma_{t/e} = \begin{bmatrix} \psi_t \\ \theta_t \\ \phi_t \end{bmatrix} = \begin{bmatrix} \pi/10 \\ 0 \\ \pi/6 \end{bmatrix} \text{ rad}$$

In this work, the rotation convention is fixed as yaw-pitch-roll (YPR),

$${}^e{}_tR = R_z(\psi_t) R_y(\theta_t) R_x(\phi_t), \quad (2)$$

with the elementary rotations explicitly given by:

$$R_z(\psi_t) = \begin{bmatrix} \cos \psi_t & -\sin \psi_t & 0 \\ \sin \psi_t & \cos \psi_t & 0 \\ 0 & 0 & 1 \end{bmatrix}, \quad R_y(\theta_t) = \begin{bmatrix} \cos \theta_t & 0 & \sin \theta_t \\ 0 & 1 & 0 \\ -\sin \theta_t & 0 & \cos \theta_t \end{bmatrix}, \quad R_x(\phi_t) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos \phi_t & -\sin \phi_t \\ 0 & \sin \phi_t & \cos \phi_t \end{bmatrix} \quad (3)$$

Therefore, the constant tool rotation ${}^e{}_tR$ is obtained by the product in (2), and the complete constant tool transform is

$${}^e{}_tT = \begin{pmatrix} {}^e{}_tR & {}^e r_{e/t} \\ 0 & 1 \end{pmatrix} \quad (4)$$

Let ${}^b{}_eT(q)$ denote the end-effector pose obtained by forward kinematics from the chain of link transforms. Since the tool is rigidly attached to the end-effector, the tool pose in the base frame is computed by direct composition:

$${}^b{}_tT(q) = {}^b{}_eT(q) {}^e{}_tT \quad (5)$$

To support this extension, the class `geometricModel` was augmented with a new method `getToolTransformWrtBase()`. The method outputs the homogeneous transformation ${}^b{}_tT(q)$ for the current joint configuration q stored in the geometric model. Internally, it:

- (i) retrieves the end-effector transform ${}^b{}_eT(q)$ using the already available forward-kinematics method, and
- (ii) post-multiplies it by the constant tool transform ${}^e{}_tT$, thus implementing (5).

To provide a numerical confirmation of the proposed modelling steps, the constant tool transform ${}^e{}_tT$ was first computed as described in (2) - (4), by constructing ${}^e{}_tR = R_z(\psi_t)R_y(\theta_t)R_x(\phi_t)$ from the assigned YPR angles and by using the translation ${}^e r_{e/t} = [0.3 \ 0.1 \ 0]^T$ m. The resulting matrix is:

$${}^e{}_tT = \begin{bmatrix} 0.9511 & -0.2676 & 0.1545 & 0.3000 \\ 0.3090 & 0.8236 & -0.4755 & 0.1000 \\ 0 & 0.5000 & 0.8660 & 0 \\ 0 & 0 & 0 & 1.0000 \end{bmatrix}$$

Then, by calling the newly introduced method `getToolTransformWrtBase()`, the tool pose with respect to the base frame was obtained according to (5). For the current configuration, the returned tool transform is:

$${}^b{}_tT(q) = \begin{bmatrix} -0.0000 & 0.5000 & 0.8660 & -0.0000 \\ -0.4540 & 0.7716 & -0.4455 & -1.1369 \\ -0.8910 & -0.3932 & 0.2270 & 0.2327 \\ 0 & 0 & 0 & 1.0000 \end{bmatrix}$$

This result confirms that the tool frame has been correctly integrated into the geometric model of the manipulator.

3 Exercise 2

3.1 Q2.1

The objective is to compute the Cartesian error between the current tool pose b_tT and the desired goal pose b_gT . The goal frame is defined by the position:

$${}^bO_g = \begin{bmatrix} 0.2 \\ -0.7 \\ 0.3 \end{bmatrix} \text{ m},$$

and by the yaw-pitch-roll angles:

$$\Gamma_g = \begin{bmatrix} \psi_g \\ \theta_g \\ \phi_g \end{bmatrix} = \begin{bmatrix} 0 \\ 1.57 \\ 0 \end{bmatrix} \text{ rad}$$

Using the same YPR convention adopted for the tool frame, the goal rotation is constructed as:

$${}^b_gR = R_z(\psi_g) R_y(\theta_g) R_x(\phi_g), \quad (6)$$

and the corresponding homogeneous transformation is:

$${}^b_gT = \begin{pmatrix} {}^b_gR & {}^bO_g \\ 0 & 1 \end{pmatrix} \quad (7)$$

In the implementation, this construction yields the numerical matrix:

$${}^b_gT = \begin{bmatrix} 0.0008 & 0 & 1.0000 & 0.2000 \\ 0 & 1.0000 & 0 & -0.7000 \\ -1.0000 & 0 & 0.0008 & 0.3000 \\ 0 & 0 & 0 & 1.0000 \end{bmatrix}$$

Let ${}^b_tT = \begin{pmatrix} {}^b_tR & {}^bO_t \\ 0 & 1 \end{pmatrix}$ be the current tool pose obtained from forward kinematics. The translational error is defined as:

$$e_p = {}^bO_g - {}^bO_t. \quad (8)$$

For the rotational component, the error is computed through the relative rotation that maps the tool frame to the goal frame. From b_tR and b_gR , the relative rotation expressed in the tool frame is:

$${}^t_gR = ({}^b_tR)^\top {}^b_gR. \quad (9)$$

The matrix t_gR is then converted into angle-axis form, defined by a unit axis $h \in \mathbb{R}^3$ and a rotation angle $\theta \in [0, \pi]$ such that:

$${}^t_gR = \exp(\theta [h \times]), \quad (10)$$

where $[h \times]$ is the skew-symmetric matrix associated with the cross product. The corresponding orientation error vector in the tool frame is:

$${}^t_{e_o} = \theta h. \quad (11)$$

Since the control law is formulated in the base frame, the orientation error is finally expressed in $\langle b \rangle$ by applying the current tool rotation:

$$e_o = {}^b_tR {}^t_{e_o} = {}^b_tR(\theta h) \quad (12)$$

The full Cartesian error is therefore:

$$e = \begin{bmatrix} e_p \\ e_o \end{bmatrix} \in \mathbb{R}^6 \quad (13)$$

Using (8)-(13), the obtained Cartesian error is:

$$e = \begin{bmatrix} 0.2000 \\ 0.4369 \\ 0.0673 \\ 0.4601 \\ 0.1226 \\ 0.5140 \end{bmatrix}$$

For completeness, the orientation error was also computed using the method reported in Siciliano; this alternative formulation is not developed here. The two results share the same position error, while small differences appear in the rotational components, which is expected since the two methods rely on different local parameterizations of the orientation mismatch. In this case, the difference between the two orientation error vectors is:

$$\Delta e_o = \begin{bmatrix} -0.0367 \\ -0.0098 \\ -0.0410 \end{bmatrix}$$

To ensure numerical robustness in control, the conversion from rotation matrix to angle-axis was implemented with safeguards: the argument of $\arccos(\cdot)$ is clamped to $[-1, 1]$, the small-angle case returns a zero error vector, and the $\theta \approx \pi$ case uses the $(R + I) * 0.5$ construction to extract a stable rotation axis. These modifications prevent jumps in h and θ due to floating-point effects and improve behaviour near singular configurations of the angle-axis representation.

3.2 Q2.2

The second step of the inverse-kinematics control consists in generating the desired Cartesian reference twist of the tool with respect to the base frame. In this work, the reference is obtained through a proportional Cartesian law that maps the Cartesian error computed in 3.1 into linear and angular reference velocities:

$${}^b\dot{x}^* = \begin{bmatrix} {}^b v_{t/b}^* \\ {}^b \omega_{t/b}^* \end{bmatrix} = \begin{bmatrix} K_L e_p \\ K_A e_o \end{bmatrix}, \quad (14)$$

where $e_p \in \mathbb{R}^3$ is the position error, $e_o \in \mathbb{R}^3$ is the orientation error expressed in the base frame (angle-axis), and the gains are selected as diagonal matrices:

$$K_L = \kappa_l I_3, \quad K_A = \kappa_a I_3, \quad (15)$$

with $\kappa_l = 0.8$ and $\kappa_a = 0.8$.

The proportional Cartesian controller can also be written in a matrix form, which ensures the independent regulation of linear and angular components:

$${}^b\nu_{t/b}^* = \begin{bmatrix} \kappa_l I_3 & 0 \\ 0 & \kappa_a I_3 \end{bmatrix} {}^b e \quad (16)$$

Given the adopted stacking $[v; \omega]$, (16) is equivalent to ${}^b v_{t/b}^* = \kappa_l e_p$ and ${}^b \omega_{t/b}^* = \kappa_a e_o$, and the reference twist is returned as ${}^b\nu_{t/b}^* = [{}^b v_{t/b}^* \quad {}^b \omega_{t/b}^*]^\top$.

The computation is encapsulated in the class `cartesianControl` through the method `getCartesianReference(bTg)`. At each control iteration, the current tool pose ${}^b_t T$ is retrieved from the geometric model via `getToolTransformWrtBase()`. The linear error is computed as the difference between the goal and the current tool position, $e_p = {}^b O_g - {}^b O_t$. The orientation error is computed using the angle-axis approach: from the relative rotation ${}^t_g R = ({}^b_t R)^\top {}^b_g R$, `RotToAngleAxis` returns an axis h and an angle θ , producing the tool-frame error vector ${}^t e_o = \theta h$, which is then expressed in the base frame as $e_o = {}^b_t R {}^t e_o$. Finally, the proportional law (14) is applied and the reference twist is stacked according to the adopted convention $[v; \omega]$:

$${}^b\dot{x}^* = \begin{bmatrix} {}^b v_{t/b}^* \\ {}^b \omega_{t/b}^* \end{bmatrix}$$

With $\kappa_l = \kappa_a = 0.8$, the computed reference velocities are:

$${}^b \omega_{t/b}^* = \begin{bmatrix} 0.3681 \\ 0.0981 \\ 0.4112 \end{bmatrix}, \quad {}^b v_{t/b}^* = \begin{bmatrix} 0.1600 \\ 0.3496 \\ 0.0539 \end{bmatrix}$$

These values represent the desired angular and linear tool velocities that drive the tool frame toward the goal frame under the proportional Cartesian control law. In the next step, this Cartesian reference twist will be mapped into joint velocities through Jacobian-based inverse kinematics.

3.3 Q2.3

Once the Cartesian reference twist of the tool has been obtained in 3.2, the desired joint velocities $\dot{\vec{q}}$ are computed through differential inverse kinematics. Since the objective is to control the tool frame, the tool Jacobian ${}^b J_t(q) \in \mathbb{R}^{6 \times 7}$ is employed, relating joint velocities to the tool twist in the base frame according to:

$${}^b \dot{x} = {}^b J_t(q) \dot{q}, \quad (17)$$

where ${}^b \dot{x} = [{}^b v_{t/b} \ {}^b \omega_{t/b}]^\top$ follows the adopted stacking $[v; \omega]$. By imposing ${}^b \dot{x} = {}^b \dot{x}^*$, the inverse kinematics problem becomes the computation of $\dot{\vec{q}}$ such that:

$${}^b J_t(q) \dot{\vec{q}} = {}^b \dot{x}^* \quad (18)$$

Because the manipulator is redundant (7 joints for a 6-D task), (18) admits infinitely many solutions; in this work the minimum-norm solution is selected through the pseudoinverse of the Jacobian.

The tool Jacobian ${}^b J_t(q)$ is computed from the geometric model by exploiting the fact that each joint axis is aligned with the local z -axis. For the j -th joint, the corresponding Jacobian column is formed from the joint axis z_j and origin O_j expressed in the base frame, and from the tool origin O_t :

$$(\text{revolute}) \quad J_t^{(j)} = \begin{bmatrix} z_j \times (O_t - O_j) \\ z_j \end{bmatrix}, \quad (\text{prismatic}) \quad J_t^{(j)} = \begin{bmatrix} z_j \\ 0 \end{bmatrix}$$

To obtain a numerically reliable inverse near singular configurations, the pseudoinverse is computed via singular value decomposition. Let:

$${}^b J_t = U \Sigma V^\top, \quad (19)$$

where $U \in \mathbb{R}^{6 \times 6}$ and $V \in \mathbb{R}^{7 \times 7}$ are orthogonal matrices and $\Sigma \in \mathbb{R}^{6 \times 7}$ collects the singular values σ_i . The minimum-norm solution is then:

$$\dot{\vec{q}} = {}^b J_t^\# {}^b \dot{x}^*, \quad {}^b J_t^\# = V \Sigma^\# U^\top, \quad (20)$$

where $\Sigma^\#$ is obtained by inverting the non-negligible singular values and discarding the remaining ones according to a numerical tolerance, which acts as an SVD approximation.

For additional robustness, a damped least-squares inversion is used to mitigate numerical issues close to kinematic singularities. The damped joint-velocity command is computed as:

$$\dot{\vec{q}} = {}^b J_t(q)^\top ({}^b J_t(q) {}^b J_t(q)^\top + \lambda^2 I_6)^{-1} {}^b \dot{x}^*, \quad (21)$$

which is equivalent to the SVD damping rule $\sigma_i^{-1} \mapsto \sigma_i / (\sigma_i^2 + \lambda^2)$. In the implementation, a constant damping $\lambda = 0.01$ is adopted.

Evaluating the above at the initial configuration yields the joint-velocity commands:

$$\dot{\vec{q}}_\lambda = \begin{bmatrix} -0.5123 \\ 0.1553 \\ 1.3043 \\ -0.0701 \\ -1.0206 \\ -0.1988 \\ 0.4532 \end{bmatrix}, \quad \dot{\vec{q}}_{\text{svd}} = \begin{bmatrix} -0.5197 \\ 0.1553 \\ 1.3164 \\ -0.0701 \\ -1.0289 \\ -0.1988 \\ 0.4533 \end{bmatrix}$$

The difference between the two joint velocity vectors is:

$$\Delta \dot{\vec{q}} = \dot{\vec{q}}_{\text{svd}} - \dot{\vec{q}}_\lambda = \begin{bmatrix} -0.0074 \\ 0.0001 \\ 0.0121 \\ 0.0000 \\ -0.0084 \\ 0.0000 \\ 0.0001 \end{bmatrix},$$

and its small magnitude confirms the numerical consistency of the pseudoinverse and damped SVD-based implementations.

3.4 Q2.4

The simulation implements a discrete-time integration loop to evolve the manipulator's state from the initial configuration to the goal. The simulation itself is made up of three important phases:

i) Discrete Time Integration

A first-order Euler integration scheme was utilized to approximate the continuous joint trajectories. At each time step t_k , given the current joint configuration $q(t_k)$ and the computed joint velocities $\dot{q}(t_k)$ (derived from the Inverse Kinematics solver using the Damped Least Squares method used at 3.3), the next state is projected as:

$$q(t_{k+1}) = q(t_k) + \dot{q}(t_k) \cdot \Delta t$$

This linear approximation is valid for sufficiently small time steps Δt .

ii) Saturation and Joint Limits

Real-world manipulators operate within specific mechanical ranges. As a result, the integration step includes a saturation filter to ensure the geometric validity of the model. Hard constraints $[q_{min}, q_{max}]$ are enforced on the updated joint angles. Therefore, if the integrated value for the i -th joint exceeds its bound, it is clamped:

$$q_i(t_{k+1}) = \begin{cases} q_{i,min} & \text{if } q_i^* < q_{i,min} \\ q_{i,max} & \text{if } q_i^* > q_{i,max} \\ q_i^* & \text{otherwise} \end{cases}$$

iii) Closed-Loop Updates

The updated q vector is immediately fed back into the *geometricModel*. This updates the Jacobian and the forward kinematics for the subsequent iteration, closing the control loop and allowing the *cartesianControl* to re-evaluate the error vector e based on the new actual position.

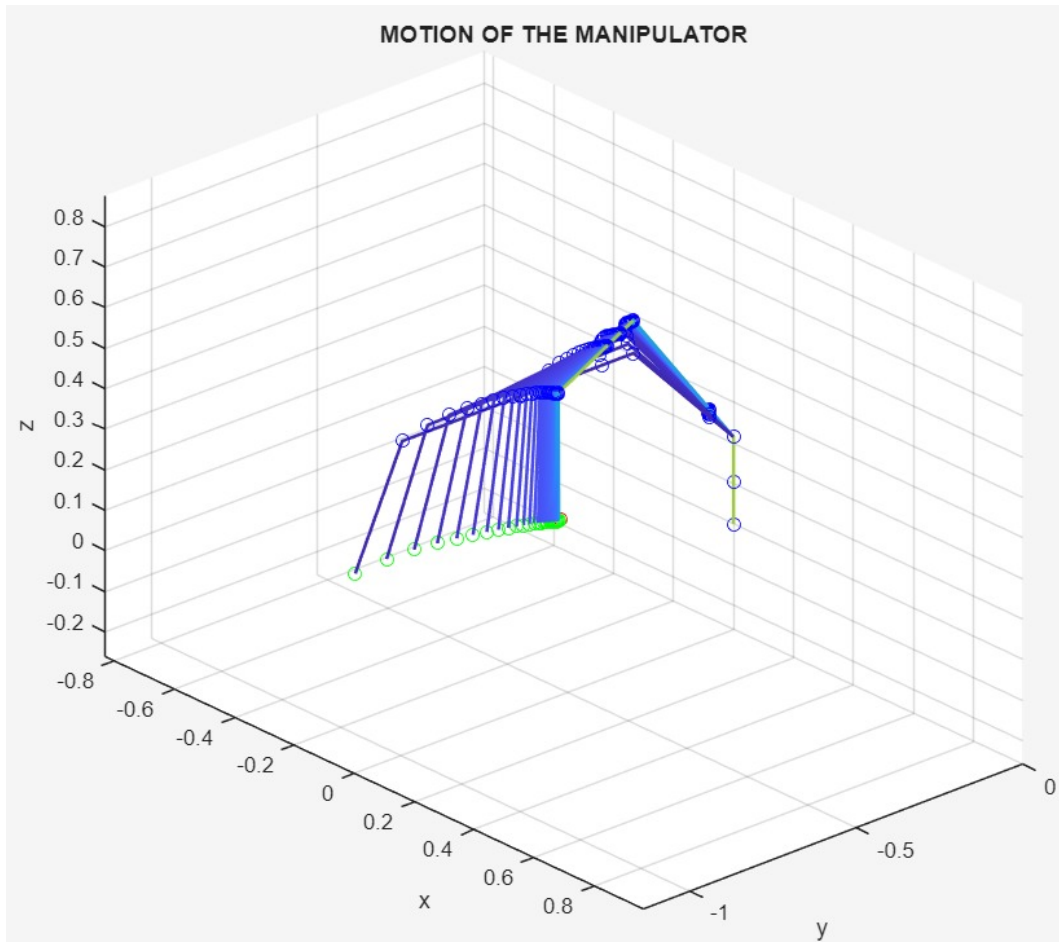


Figure 1: The path that the manipulator takes during the simulation.

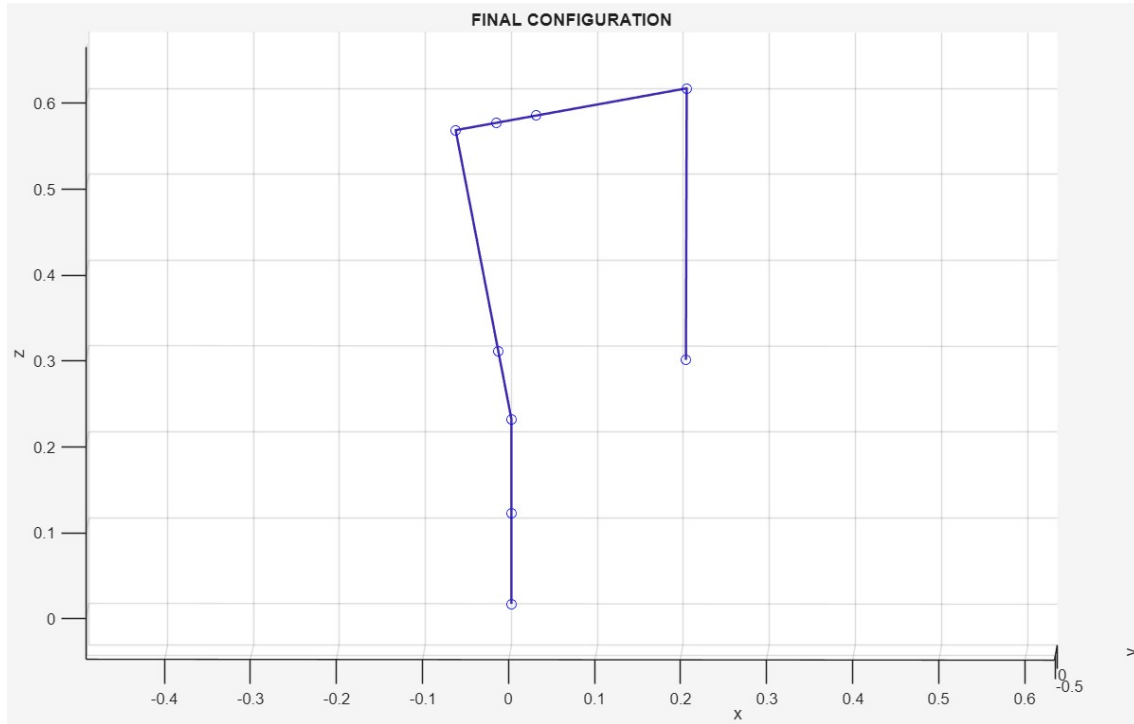


Figure 2: The final configuration of the manipulator when it reaches the goal.

Notes on the simulation

The manipulator reaches the goal only if the time of the simulation is increased from 10s to 20s. This behavior is dictated by the convergence rate of your control law relative to the physical distance the robot must travel. The control law:

$$\nu = K \cdot e$$

imposes a specific velocity profile, due to the fact that the robot moves to a speed proportional to the error and the gains defined; and not as fast as possible. Since the speed at which the robot closes this gap is determined by the gains and the kinematic constraints, changing the simulation duration does not make the robot faster. It only changes the time that the robot is "allowed" to move. Another aspect of it is the exponential decay of the error. The latter requires a fixed amount of physical time to be reduced from the initial value to the tolerance threshold.

3.5 Q2.5

The analysis of the evolution of the velocity of the end-effector in Figure 3.5 offers critical insight into the performance of the kinematic solver. It reveals a divergence between the rotational and translational behaviors. The angular velocity components ($\omega_x, \omega_y, \omega_z$) exhibit a stable transient response, successfully decaying to zero at approximately $t = 8s$. As a result this confirms that the orientation error is eliminated and the tool frame aligns with the target rotation.

On the other hand the linear velocity profiles ($\dot{x}, \dot{y}$) in the plot do not converge to zero but instead stabilize at significant constant values. This plateau is characteristic of joint limit saturation in a proportional control scheme: the manipulator mechanically hits a joint constraint (q_{min} or q_{max}), physically halting motion, while the controller continues to generate a constant velocity command, due to the persistent position error.

This saturation behavior observed in the graphical data stands in contrast to the final numerical results obtained in Exercise 2.5. The numerical log indicated successful convergence, with linear velocities reducing to negligible magnitudes and angular velocities reaching absolute zero.

$$\text{End-effector velocity } [v; \omega] = \begin{bmatrix} -0.0049 \\ 0.0086 \\ -0.0011 \\ 0.0000 \\ 0.0000 \\ 0.0000 \end{bmatrix} \quad \text{Tool velocity } [v; \omega] = \begin{bmatrix} -0.0049 \\ 0.0086 \\ -0.0011 \\ 0.0000 \\ 0.0000 \\ 0.0000 \end{bmatrix}$$

The observed discrepancy between the graphical representation and the numerical data is a result of the normalization process, not a calculation error. The plot displays normalized velocity vectors,

$$\frac{\mathbf{v}}{\|\mathbf{v}\|},$$

to visualize directionality rather than magnitude. Consequently, the plotted unit vectors appear significantly larger than the actual physical velocities, which are quantitatively expressed by the numerical values.

For instance, the magnitude of the velocity vector is approximately 0.01 m/s, roughly two orders of magnitude smaller than the unit vectors shown in the plot:

$$\|\mathbf{v}\| = \sqrt{(-0.0049)^2 + (0.0086)^2 + (-0.0011)^2},$$

$$\|\mathbf{v}\| \approx \sqrt{9.918 \times 10^{-5}},$$

$$\|\mathbf{v}\| \approx 0.00996 \text{ m/s}.$$

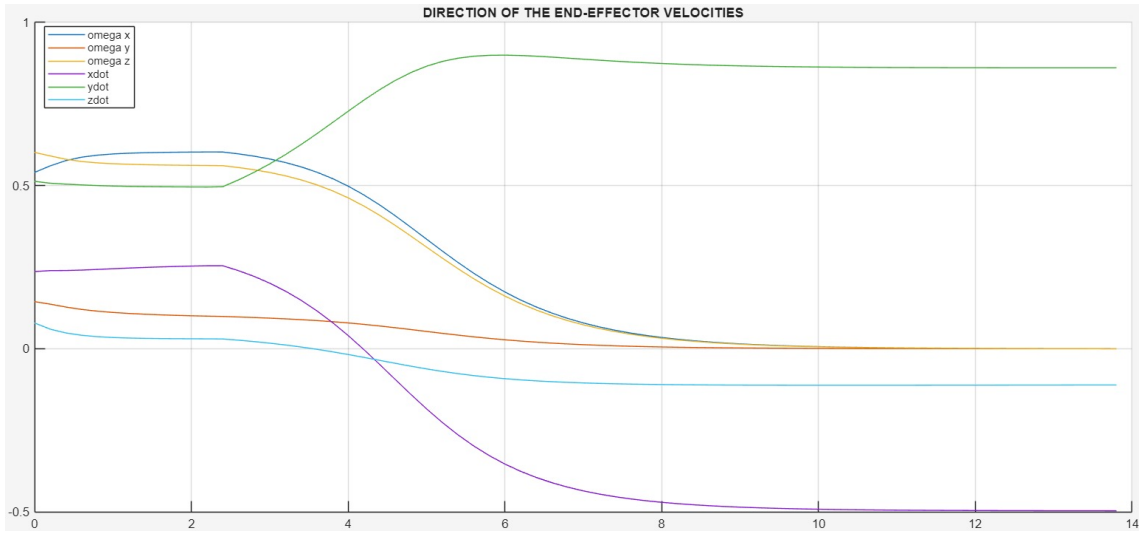


Figure 3: The velocities plotted in time.